ID 2230 Data Structures and Applications

Big O notation

Typically algorithms' performance is measured using Θ notation or O notation

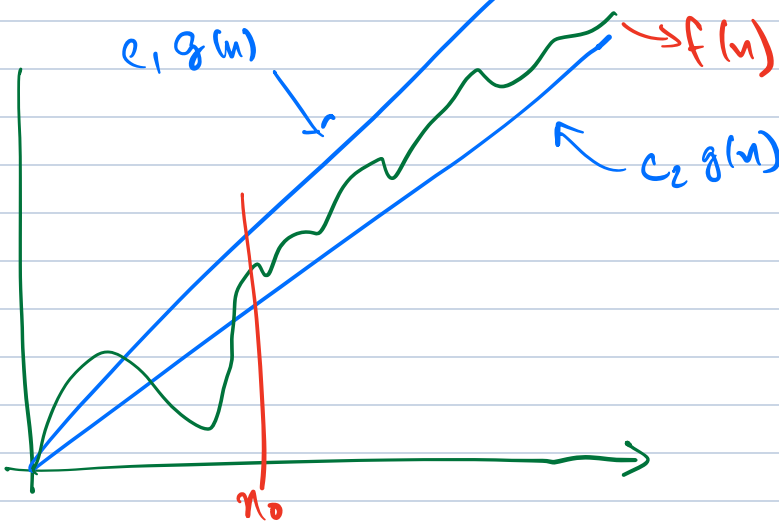
$O(g(n)) = \{ f(n)$: there exists c, n_0 such that $0 \leq f(n) \leq c\,g(n)$ for all $n \geq n_0 \}$.

Q: Say $g_1(n) \geq g(n)$, and $f(n) = O(g(n))$ does it follow that $f(n) = O(g_1(n))$ also?

A: YES: Same n_0 and c would suffice

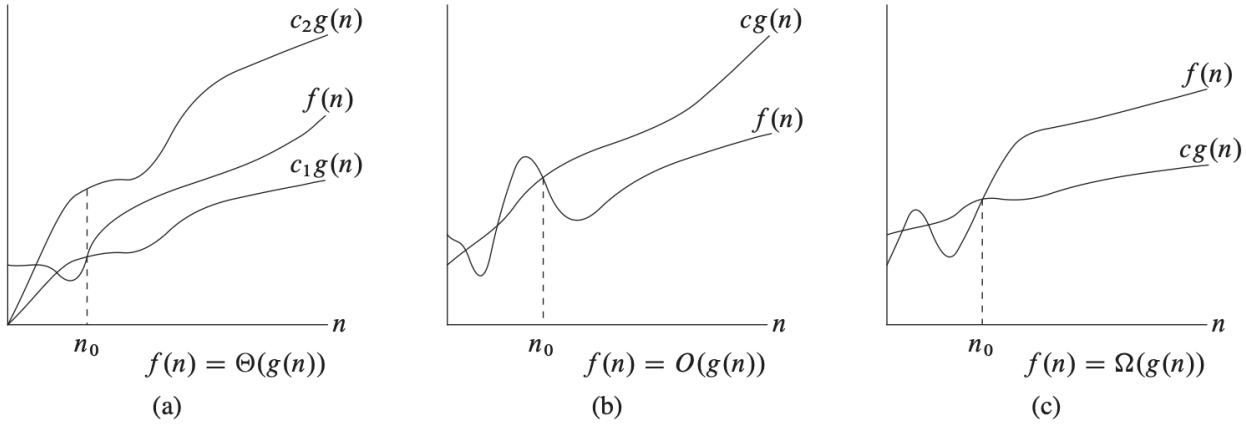
Ω notation: $f(n) = \Omega(g(n))$ if there is a c and n_0 such that for $n \geq n_0$, $f(n) \geq c\, g(n)$.

Θ notation: $f(n) = \Theta(g(n))$ if there exist $c_1 > c_2 > 0$ and n_0 such that for $n \geq n_0$, $c_1 g(n) \geq f(n) \geq c_2 g(n)$.



Review: O notation → Chap 3

$O, \Theta, \Omega, o, \omega$



From CLRS (Chapter 3).

$f(n) = n^2 + 2n + 5$

$f(n) = O(n^2)$.

For all $n > 4$, $f(n) \leq 2n^2$.

$n_0 = 4$, $c = 2$

$n^2 + 2n + 5 \leq 2n^2$ for $n \geq 4$

$n = 4$	29	32
$n = 5$	40	50

$f(n) = 3n^2 + 3n + 100$

Is $3n^2 + 3n + 100 = O(n^2)$? YES.

$n_0 = 10$. $c = 10$.

$3n^2 + 3n + 100 \leq 10 \cdot n^2$.

$f(n) = 3n^2 + 3n + 100$

$f(n) = \Theta(n^2)$

For All $n \geq$ (0) $= n_0$, $3n^2 + 3n + 100 \geq$ (1) n^2 (c_2 Const)

For all $n \geq 10$, $1 \cdot n^2 \leq 3n^2 + 3n + 100 \leq 10 \cdot n^2$

Verify it!

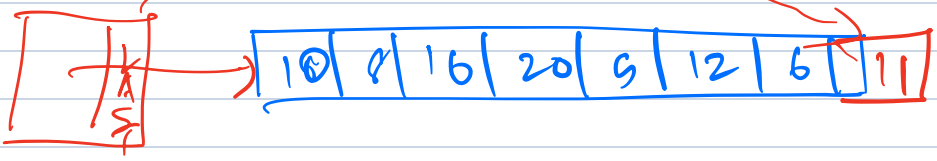
Searching / Sorting

→ Suppose we want to store data / organize data in such a way that it is easy to search, insert / delete / modify.

→ Say your bank records, airport traffic system, many applications.

→ What are the ways?

Unsorted Array



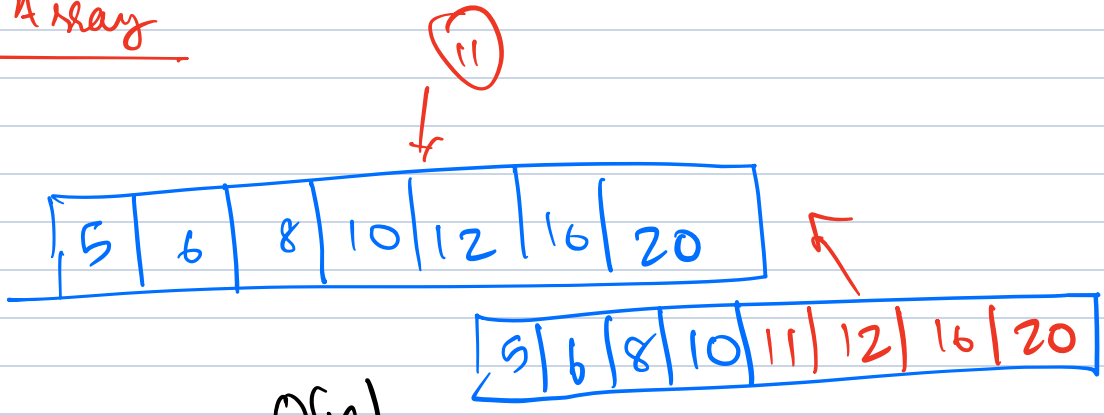
→ Easy to insert → Easy - Add at the end : O(1).

→ Searching ? → O(n) - One may have to go through all records.

→ Deleting → Can take O(n) time because we need to search.

→ Editing → Easy after you locate the element.

Sorted Array



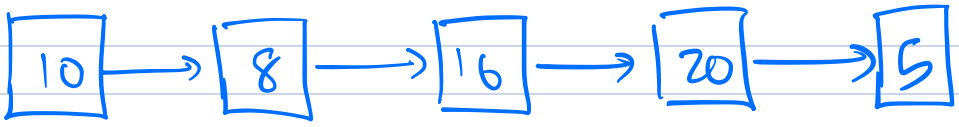
→ Insertion → O(n).
We may have to shift Ω(n) elements to make space.

→ Delete → O(n). We may have to shift Ω(n) elements to fill up the gap.

→ Edit → If we are changing the key, we may need Ω(n) time.

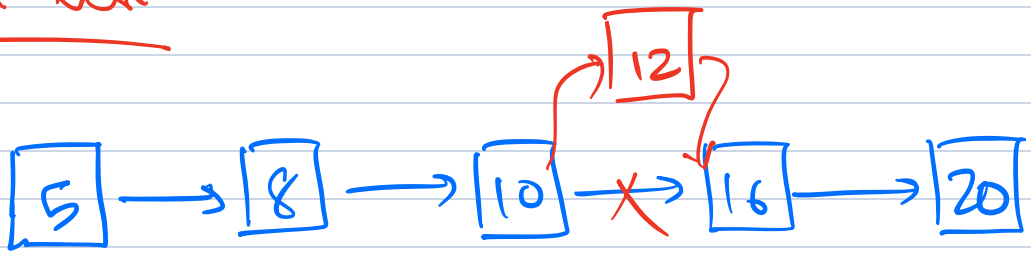
→ Search ? → BINARY SEARCH. → O(log n).

Unsorted list



→ Insert : O(1), once you find the tail.
→ Delete : Search time + O(1)
→ Search : O(n)
→ Edit : Search time + O(1).

Sorted list



→ Insert : Search + O(1).
→ Search : O(n). We cannot do binary search.
→ Delete : Search + O(1).
→ Edit : Search + Re-insert.

Binary Search

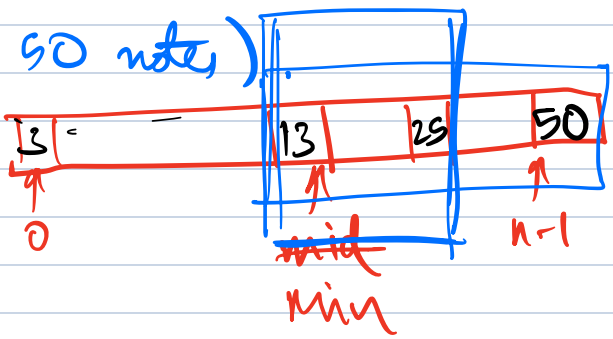
Faster Search in a sorted array.

```
1  set min = 0 and max = n - 1
2  find middle of array
3  if k is less than array[middle]
4      set max to middle - 1
5      go to line 2
6  else if k is greater than array[middle]
7      set min to middle + 1
8      go to line 2
9  else if k is equal to array[middle]
10     you found k in the array!
11 else
12     k is not in the array
```

(k: number being searched)

(line 2: $\left\lfloor \frac{min + max}{2} \right\rfloor$)

(From Harvard CS50 notes)
Courtesy.



Complexity.

In every step, the search space is cut down by half. So how many times can we divide n by 2, till we get an array of size 1 or 2?

O(log n) time ← $\leq \log_2 n$ times.

let A be the array and k be the key that is being searched.

BIN-SEARCH (A, min, max, k)

~~Set min = 0 and max = n-1~~

If either A[min] = k or A[max] = k, then return the correct entry

Else If max ≤ min + 1, then return "key not present"

mid = $\lfloor (min + max)/2 \rfloor$

If A[mid] < k, then call

BIN-SEARCH (A, mid+1, max, k)

If A[mid] > k, then call

BIN-SEARCH (A, min, mid-1, k)

This takes $O(\log n)$ time.

In every step, we are reducing the search space by half.

$$\log_2 n = \log_e n * \log_2 e$$

$$\log_2 n = \log_{10} n * \log_2 10$$

$O(\log n)$

Sorting

Input : Given an array, A with values that are not sorted

Output : Return the same values sorted in ascending order.

Bubble Sort.

10 8 16 20 5 12.

Loop 1.

8 10 16 20 5 12

& 10 16 20 5 12

& 10 16 20 5 12

& 10 16 5 20 12

& 10 16 5 12 20

After loop 1, the largest element will be at the end.

loop 2

& 10 16 5 12 20

& 10 16 5 12 20

& 10 16 5 12 20

& 10 5 16 12 20

& 10 5 12 16 20

Second largest is at the second last position

loop 3

8 10 5 12 16 20

8 10 5 12 16 20

8 5 10 12 16 20

8 5 10 12 16 20

loop 4

8 5 10 12 16 20

5 8 10 12 16 20

5 8 10 12 16 20

loop 5

5 8 10 12 16 20

5 8 10 12 16 20

Bubble sort (A). → indexed from 1 to n

For $k = 1$ to $n-1$

{ For $j = 1$ to $n-k$.

If $A[j] > A[j+1]$

Swap $A[j]$ & $A[j+1]$.

Why is this correct?

At the end of loop k, the k largest elements occupy the last k positions of the array in the correct relative ordering.

At the end of loop $k = n-1$, all the elements are in the correct order.

Time Complexity:

Outside loop runs $n-1$ times.

Each outside loop calls the inside loop $n-k$ times $\leq n$ times

Each iteration of the inside loop is a comparison plus possibly a swap, $\rightarrow O(1)$ time.

Total time $\leq n-1 * n * O(1)$

$= O(n^2)$

Instead of time, another measure of complexity used is the no. of comparisons.

Other sorting algorithms.

→ Selection Sort

→ Merge Sort ✓

→ QuickSort. ✓

→ Heap Sort → After heap.

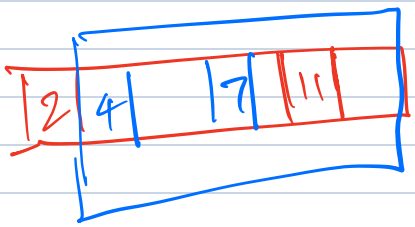
→ Insertion Sort.

→ Radix Sort.

→ Counting Sort. → n is large but range is small

→ Shell Sort.

27/Aug/2026

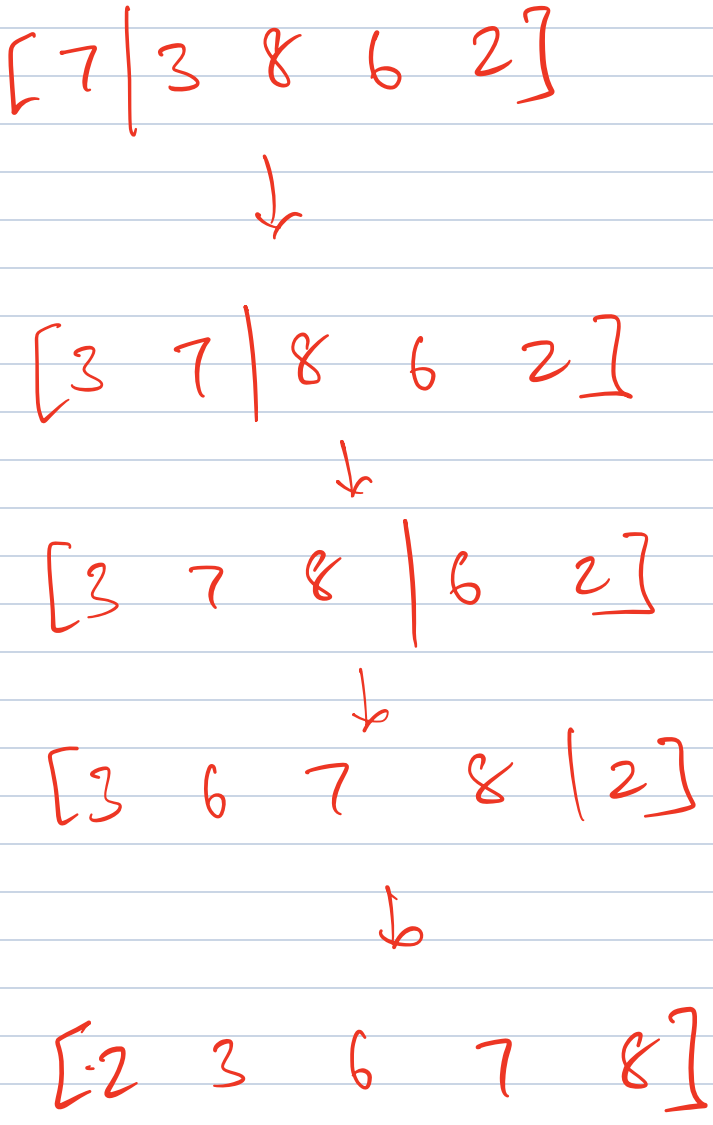


$O(n^2)$

→ Selection sort: Find smallest element and swap with the first

→ Insertion sort: Repeatedly insert the next element into the sorted

list

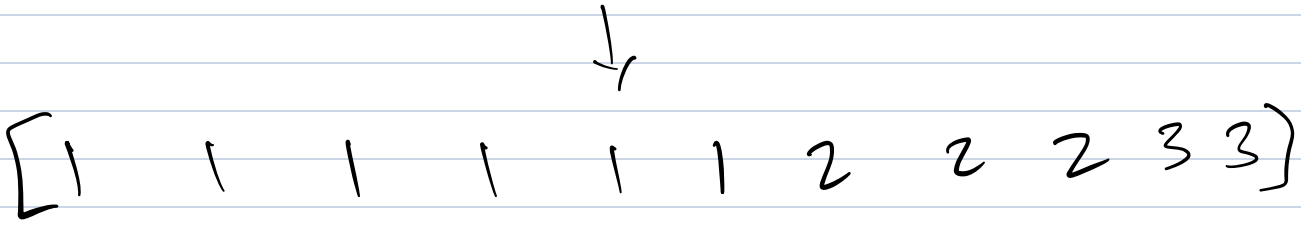


Also $O(n^2)$ running time in worst case.

→ Counting Sort: Say you have bounded range. This part is crucial.

[1 2 1 3 1 2 1 1 3 2]

$O(n+k)$



where

k is the size of the range.

Counting Sort (input, k).

Assuming the range is 1, 2, 3.. k

Count ← Array of k zeros.

Output ← Array of same length as input

For i = 1 to length (input)

If j == input[i]

Count[j] = Count[j] + 1.

Temp = 1

For i = 1 to k

For j = 1 to Count[i]

Output[Temp] = i

Temp = Temp + 1

Return Output

Counting Sort and Radix Sort are NOT COMPARISON-BASED Sorting Algorithms. In Comparison-based algorithms, you are only

→ Radix Sort: allowed to compare two input numbers.

Digit by digit sort starting from least significant digit.

91 19 55 54 37.

→ 91 54 55 37 19

→ 19 37 54 55 91

Merge Sort

Input: An array of numbers $A[1, 2 \ldots n]$

Output: A sorted version of this array

If $n > 1$

Return Merge [Mergesort $A(1, \ldots \lfloor n/2 \rfloor)$,

Mergesort $A(\lfloor n/2 \rfloor + 1, \ldots n)$]

Else

Return A. Each individually sorted.

Merge $X[1, \ldots k]$ $Y[1, \ldots l]$

If $k=0$, return $Y[1, \ldots l]$

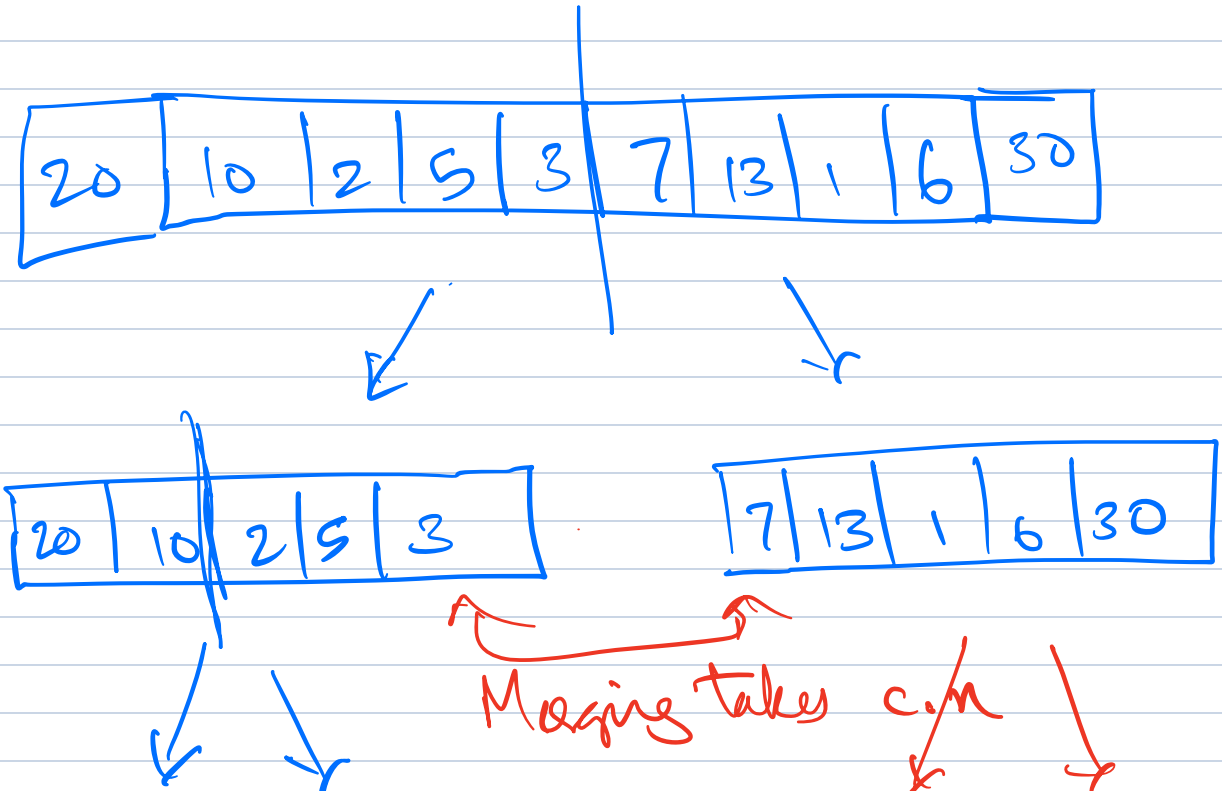
If $l=0$, return $X[1, \ldots k]$

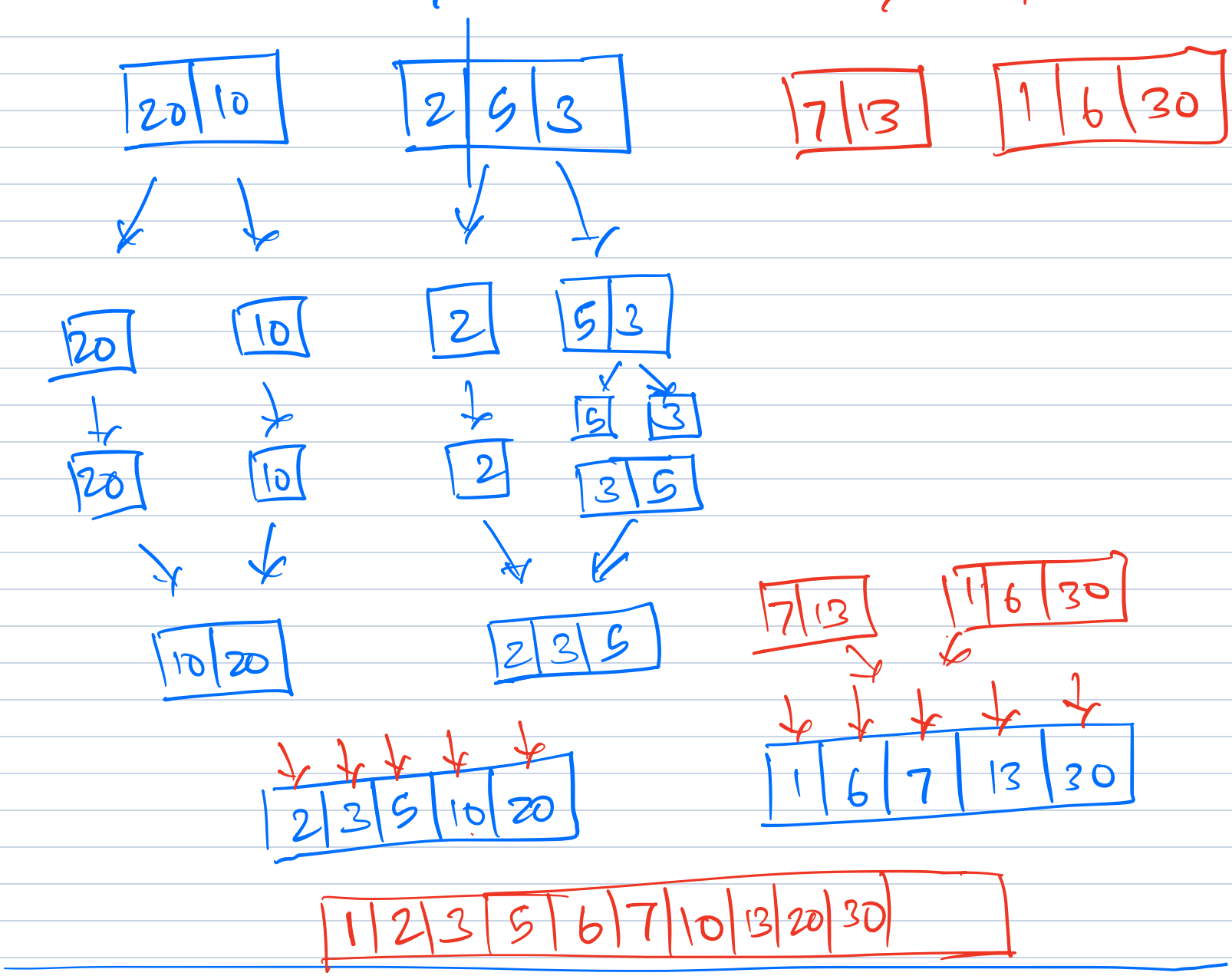
If $X[1] \leq Y[1]$

Return $X[1]$, [Merge $[X(2, \ldots k)$, $Y[1, \ldots l)]$]

Else

Return $Y[1]$, [Merge $[X(1, \ldots k)$, $Y[2, \ldots l)]$]



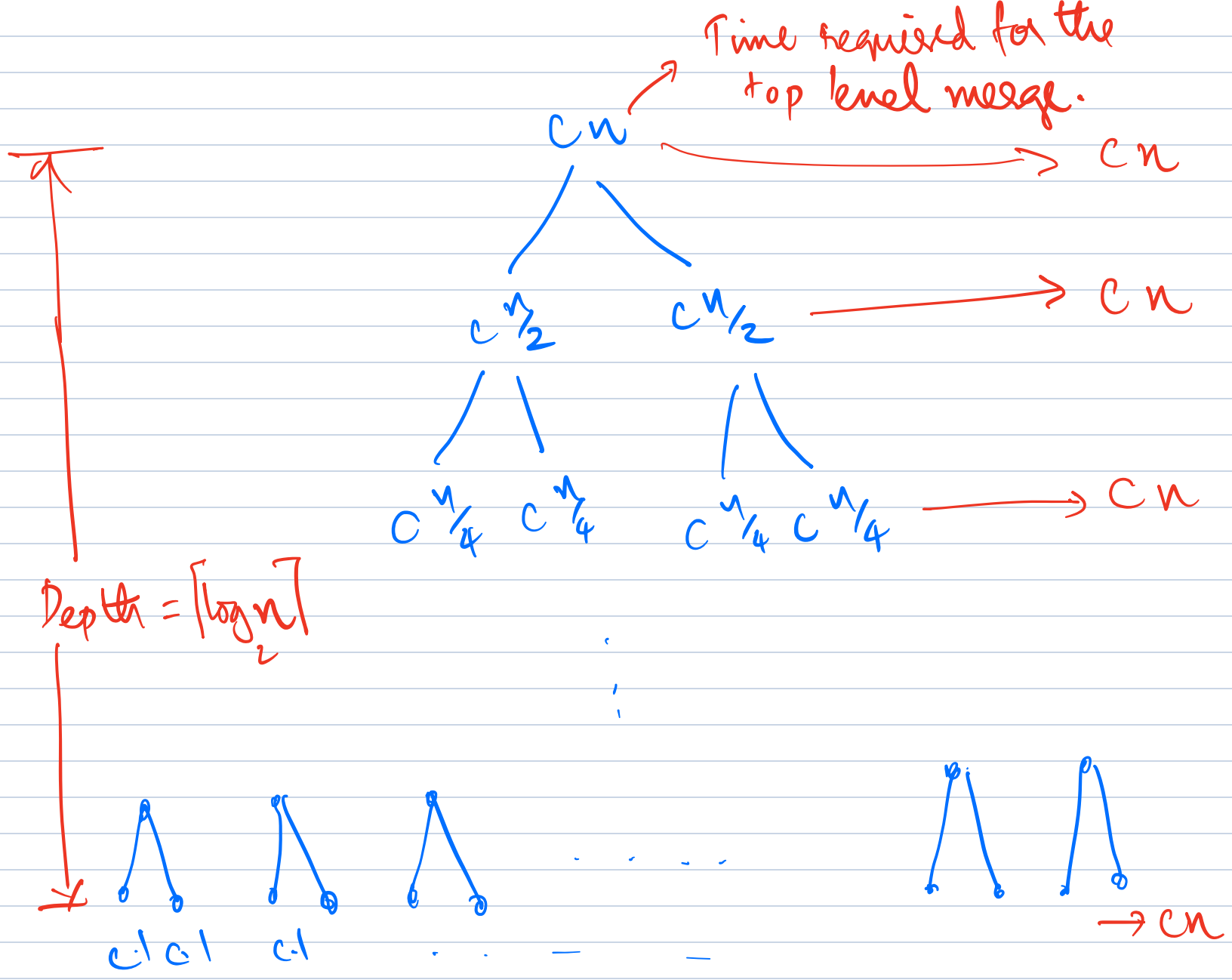


Running time for Merge : $O(k+l)$

Overall time taken for Mergesort. Let $T(n)$ represent time to solve instance of size n

Recurrence Relation $T(n) = 2T(n/2) + O(n)$.

$\Rightarrow O(n \log n)$



So the total time = effort at each level * No. of levels

$= cn * \log n$

$= cn \log n$

$= O(n \log n)$

ALGORITHMS

BY PAPADIMITRIOU
DASGUPTA
VAZIRANI

Master Theorem: If $T(n) = aT(n/b) + O(n^d)$

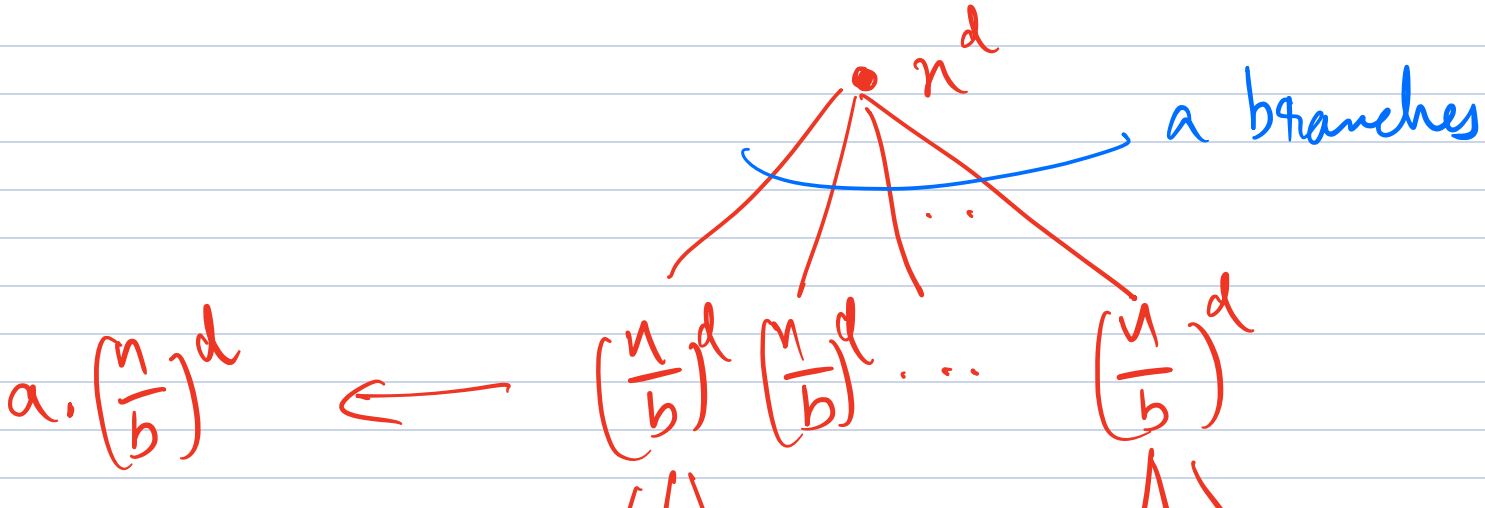
for $a > 0$, $b > 1$, $d \geq 0$, then

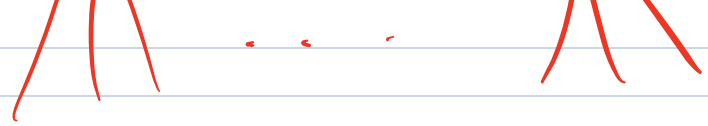
$$T(n) = \begin{cases} O(n^d) & \text{if } d > \log_b a \quad (\text{OR } b^d > a) \\ O(n^d \log n) & \text{if } d = \log_b a \quad (\text{OR } b^d = a) \\ O(n^{\log_b a}) & \text{if } d < \log_b a \quad (\text{OR } b^d < a) \end{cases}$$

Reason: Total work at k^{th} level.

$$a^k * O\left(n/b^k\right)^d = O(n^d) \cdot \left(\frac{a}{b^d}\right)^k$$

$\frac{a}{b^d} < 1 \iff d > \log_b a$:





$a^2 \left(\frac{n}{b^2}\right)^d$

If $a < b^d$, then the top part dominates $\rightarrow T(n) = O(n^d)$

If $a = b^d$, then each level contributes equally. And total $\log_b n$ levels

$$T(n) = O(n^d \log_b n)$$

If $a > b^d$, then bottom part dominates

$$T(n) = O(n^{\log_b a})$$

QUICKSORT

QUICKSORT (A, 1, n).

If $n \leq 1$, return A.

Choose a pivot p (or equivalently an index k from 1 to n)

For example, one way could be to choose the first element, i.e, A[1] to be the pivot.

Then Partition A w.r.t to the pivot p.

Order the elements in A as

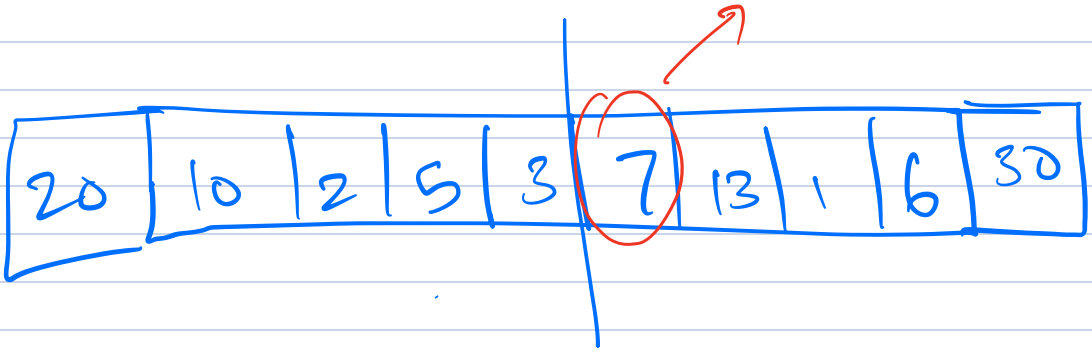
Smaller than p, p, larger than p.

Say p is now in index l.

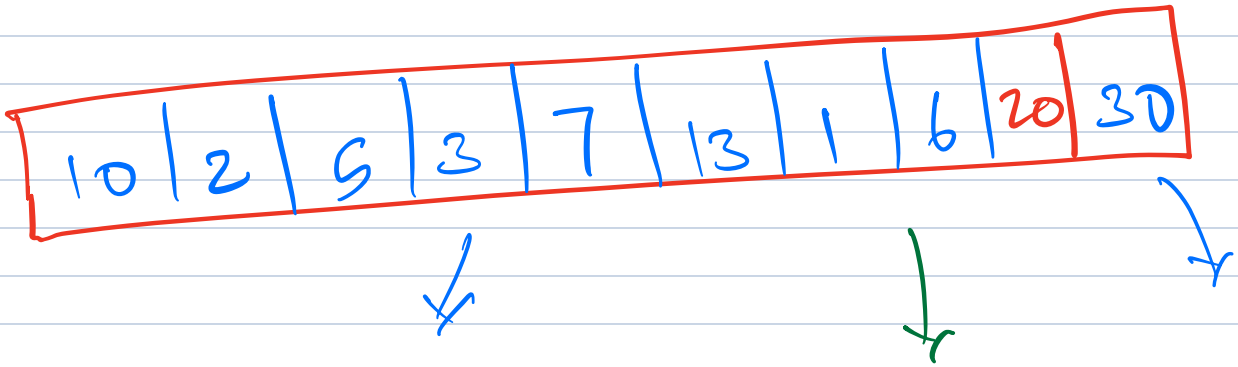
Then recursively sort left and right parts using QUICKSORT (A, 1, l-1)

and QUICKSORT (A, l+1, n)

Return A.

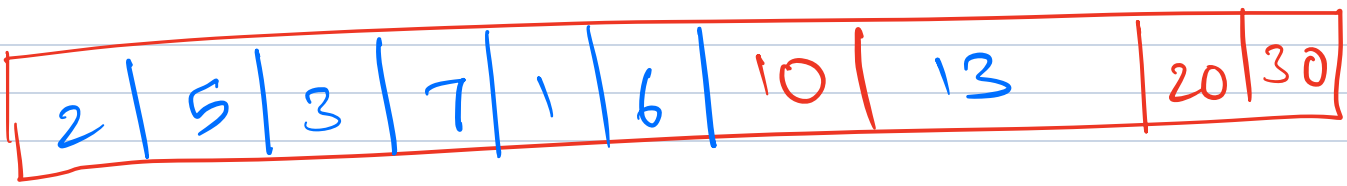


First pivot = 20.

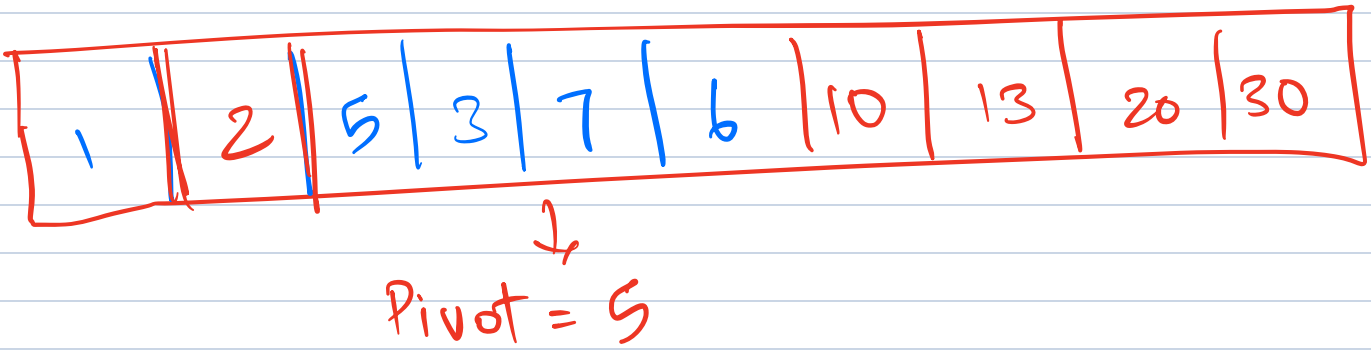


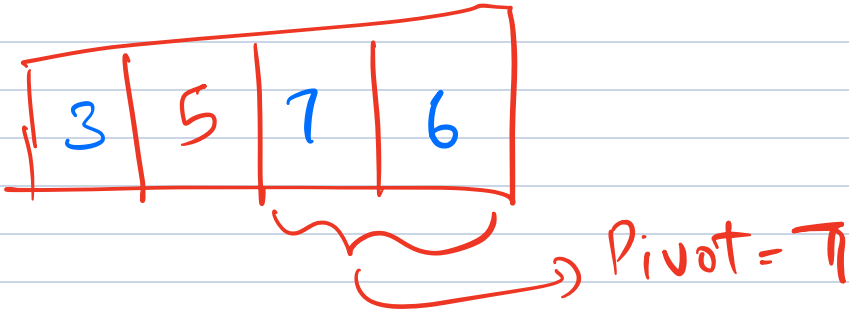
Second pivot = 10

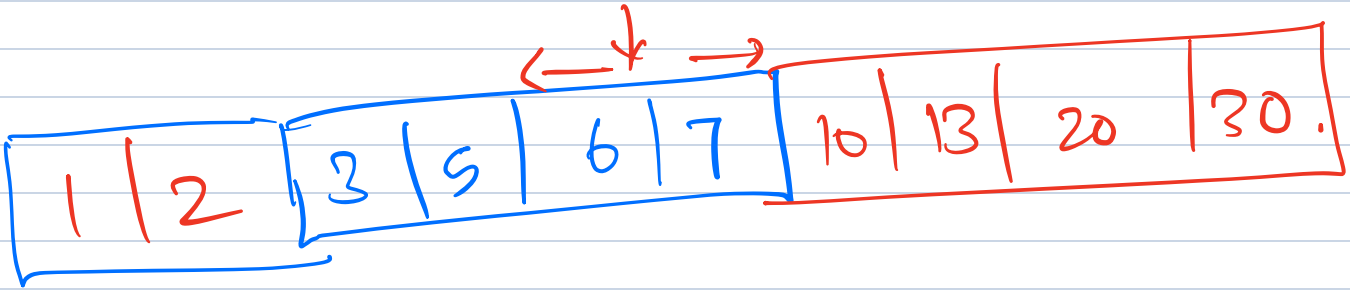
Note: After the partition operation, the pivot will be at the correctly sorted position.



For the left part, pivot = 2.

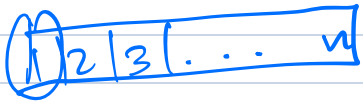






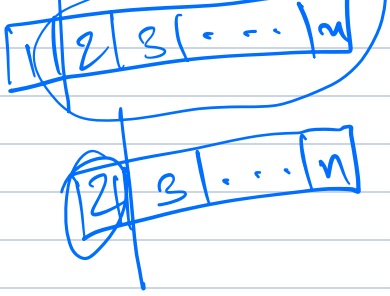
EXERCISE: Think of what happens when the input array is $[1, 2, 3, \ldots n]$ in the sorted order. And pivot rule is the first element.

↳ Running time = $O(n^2)$



↳ Memory usage = $O(n^2)$

No-of comparisons



QUICKSORT (A, p, r)

(From CLRS)

If $p < r$

$q = \text{PARTITION}(A, p, r)$ → PARTITION(·) returns q which is the index of the pivot.

QUICKSORT $(A, p, q-1)$

QUICKSORT $(A, q+1, r)$

To sort the entire array, we need to make QUICKSORT $(A, 1, n)$ as the top level call.

PARTITION (A, p, r)

$x = A[r]$ → Pivot

Pivot here is the last element $A[r]$

$i = p - 1$

For $j = p$ to $r-1$

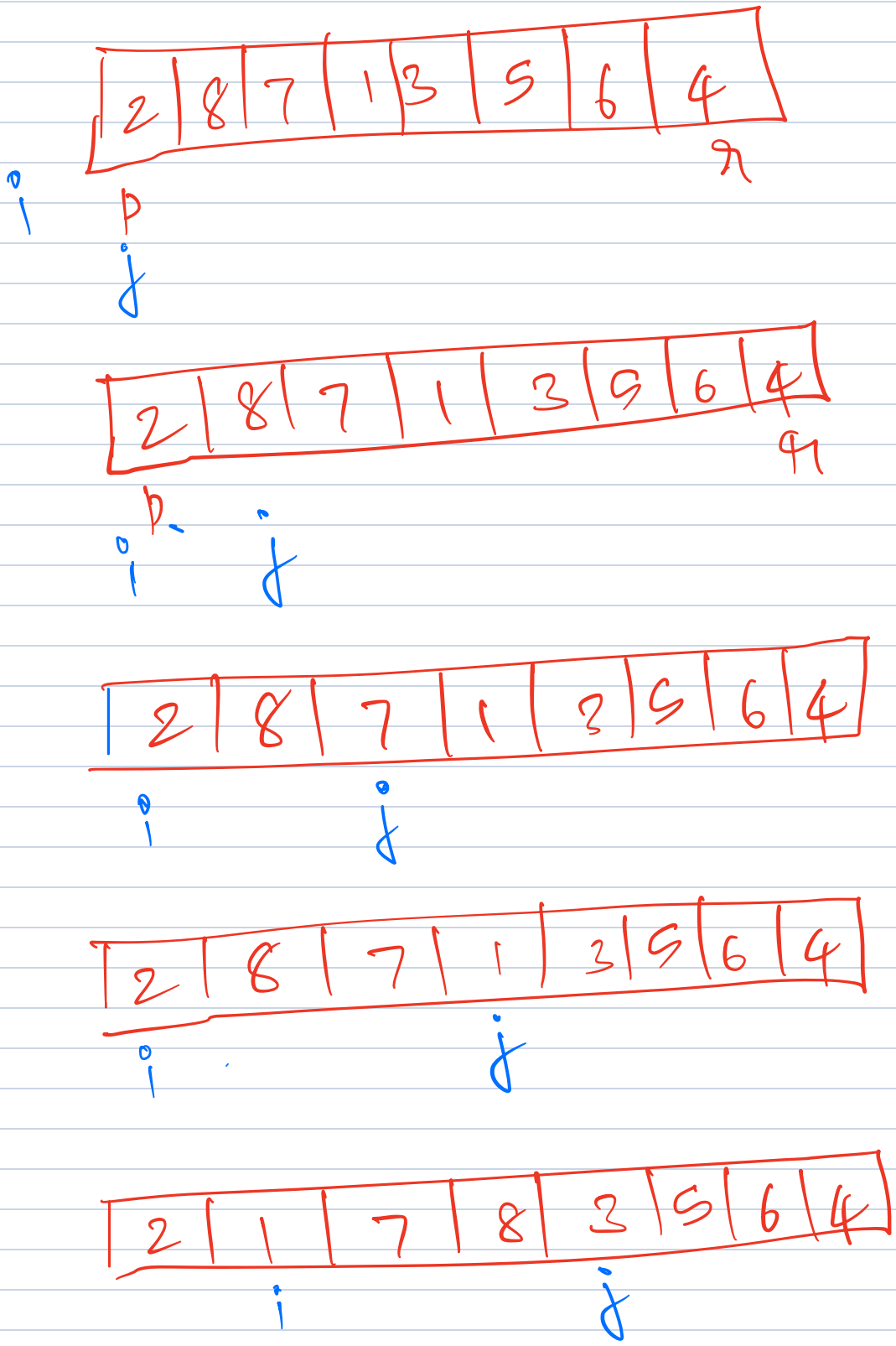
| If $A[j] \leq x$,

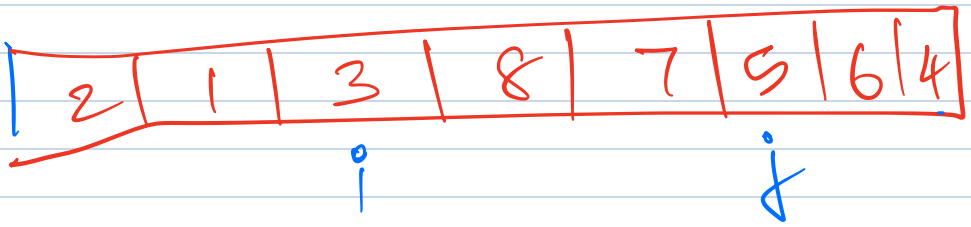
| $i = i+1$

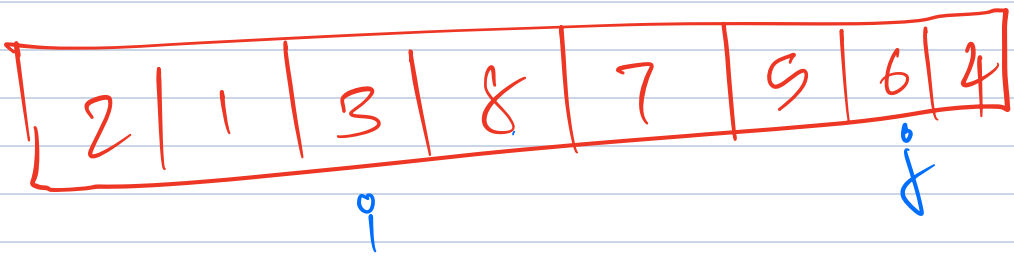
| Exchange $A[i]$ with $A[j]$

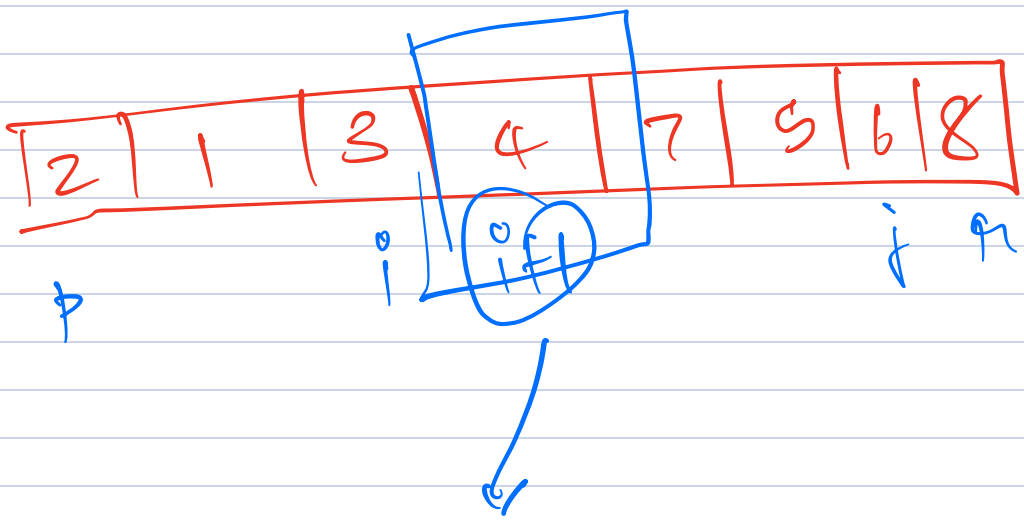
Exchange A[i+1] with A[r]

Return i+1









Return